

PROJECT REPORT
ON
HYBRID INTRUSION DETECTION SYSTEM
(SIGNATURE AND ANOMALY BASED APPROACH)

Submitted in Partial Fulfilment of Award of
BACHELOR OF TECHNOLOGY

In
Computer Science and Engineering

By
SUNKARA VENKATA RATHNAM [2021BCSE07AED449]
ALLURU LEKHYA [2021BCSE07AED399]
BHOUGYAM VENKATA SAI LIKHITHA [2021BCSE07AED391]
PYATA NITHIN KUMAR GOUD [2021BCSE07AED547]

Under the Supervision of
Dr. R. Niranchana, Assistant Professor, CSE



ALLIANCE SCHOOL OF ADVANCE COMPUTING
ALLIANCE UNIVERSITY
BENGALURU

MAY 2025



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the project work entitled “Hybrid Intrusion Detection System(Signature And Anomaly Based approach)” submitted by Sunkara Venkata Rathnam [2021BCSE07AED449], Alluru Lekhya [2021BCSE07AED399], Bougyam Venkata Sai Likhitha [2021BCSE07AED391], Pyata Nithin kumar Goud [2021BCSE07AED547] in partial fulfilment for the award of the degree of Bachelor of Technology CSE of Alliance University, is a Bonafide work accomplished under our supervision and guidance during the academic year 2024-2025. This thesis report embodies the results of original work and studies conducted by students and contents do not form the basis for the award of any other degree to the candidate or anybody else.

Dr. R. Niranchana
Assistant Professor
(Supervisor)

Dr. A Ezil Sam Leni
Professor & HOD
(Head of Department)

External Examiners

1. Name:

Signature

2. Name:

Signature



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCE COMPUTING

DECLARATION

We hereby declare that the project entitled “HYBRID INTRUSION DETECTION SYSTEM (SIGNATURE AND ANOMALY BASED APPROACH)” submitted by us in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology CSE of ASAC, Alliance University, is a record of our work carried under the supervision and guidance of Dr. R. Niranchana, Assistant Professor.

We confirm that this report truly represents the work undertaken as a part of our project work. This work is not a replication of work done previously by any other person. We also confirm that the contents of the report and the views contained there in have been discussed and deliberated with the faculty guide.

Name of the Student	University Registration Number	Signature
Sunkara Venkata Rathnam	2021BCSE07AED449	
Alluru Lekhya	2021BCSE07AED399	
Bhougyam Venkata Sai Likhitha	2021BCSE07AED391	
Pyata Nithin Kumar Goud	2021BCSE07AED547	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported and guided us throughout the completion of this project. This final year project has been a significant learning experience, and we are deeply thankful to everyone who contributed to its success.

First and foremost, we extend our heartfelt thanks to our's project guide, Dr. R. Niranchana, for their invaluable guidance, constant support, and insightful feedback at every stage of this project. Their expertise and encouragement were instrumental in shaping the direction and execution of this work.

We are also grateful to the Director, Dr. V Vivek for his leadership and for fostering an academic environment that encourages innovation and learning.

Additionally, We would like to thank the Head of Department, Dr. A Ezil Sam Leni, Program Co-ordinators and all the faculty members of the CSE department at Alliance University for providing a supportive learning environment and necessary resources.

A special note of thanks to my teammates and friends for their collaboration, ideas, and motivation during challenging times. I also appreciate the support and encouragement from my family, who have always been a source of strength and inspiration.

Lastly, I would like to acknowledge all the authors and researchers whose work I referred to while carrying out this project.

Sunkara Venkata Rathnam [2021BCSE07AED449] [CSE] [ASAC]

Alluru Lekhya [2021BCSE07AED399] [CSE] [ASAC]

Bhougyam Venkata Sai Likhitha [2021BCSE07AED391] [CSE] [ASAC]

Pyata Nithin Kumar Goud [2021BCSE07AED547] [CSE] [ASAC]

PREFACE

This project, titled "HYBRID INTRUSION DETECTION SYSTEM (SIGNATURE AND ANOMALY BASED APPROACH)," explores the integration of traditional signature-based detection methods with modern machine learning-driven anomaly detection techniques to enhance cybersecurity monitoring in networked environments. It focuses on developing a hybrid intrusion detection system capable of identifying both known and previously unseen threats through real-time network traffic analysis.

The project leverages artificial intelligence (AI) and packet analysis tools to build a comprehensive security framework. Signature-based detection is implemented using predefined rules to identify well-known attack patterns, while anomaly detection utilizes statistical and machine learning models to flag deviations from normal traffic behavior. This dual-layered approach ensures more robust and accurate threat detection, minimizing false positives and enhancing response capabilities.

The objectives of this project are to design and implement a scalable hybrid IDS, automate traffic analysis, and demonstrate effective detection of both signature-based and anomalous network behaviors. This document outlines the methodology, system components, implementation details, evaluation results, and practical applications of the proposed security solution.

ABSTRACT

With today's interconnected world of the internet, cybersecurity threats are getting more and more sophisticated, hitting network infrastructures as well as endpoint systems. To combat this increasing threat, this project introduces the design and development of a Hybrid Intrusion Detection System (HIDS) that leverages the strengths of both signature-based and anomaly-based detection approaches. The system is able to monitor in real-time network traffic as well as identifying host-level anomalies, thus having a holistic approach towards intrusion detection.

The hybrid model is deployed in Python with the aid of libraries like Scapy to capture packets and pandas for data manipulation. Network packets are captured and relevant features—like packet size, inter-arrival time, source and destination ports, TCP flags, and packet rate—are extracted. These features are used as inputs to both the signature matching and machine learning-based anomaly detection algorithms. The system architecture is modular and involves stages like environment setup, data preprocessing and collection, model training, signature definition, and real-time detection and monitoring.

In the signature-based module, the system compares captured traffic to an established set of rules modeling known attack patterns, for example, SYN flood activity or unusual port usage. Concurrently, the anomaly detection module applies a learned machine learning model to determine out-of-pattern behavior indicative of zero-day attacks or new intrusion attempts. With this dual-layered approach, the system can both accurately detect known threats and unknown threats.

In addition, the system has a monitoring and deployment feature that facilitates real-time, continuous traffic analysis and instantaneous logging of any threats detected. The outcomes of the detection process are logged and saved for further analysis, allowing security analysts to analyze threats and respond accordingly. This project proves the viability of hybrid intrusion detection through the combination of real-time analysis of network traffic and smart anomaly detection methods.

LIST OF FIGURES

Figure 3.2.1: System Architecture

Figure 5.2.2: Network Traffic Capture

Figure 5.2.3: Model Training and Evaluation Output

Figure 5.2.4: Signature -Based Threat Detection Log

Figure 5.2.5: Real-Time Detection of Network Anomalies and Threats

Figure A 3.6 to A 3.13: Code Part

LIST OF ABBREVIATIONS AND SYMBOLS USED

HIDS: Hybrid Intrusion Detection System

NIDS: Network-Based Detection System

IDS: Intrusion Detection System

ML: Machine Learning

AL: Artificial Intelligence

DoS: Denial of Service

RAM: Random Access Memory

TCP/IP: Transmission Control Protocol / Internet Protocol

JSON: JavaScript Object Notation

UI: User Interface

FP: False Positive

URL: Uniform Resource Location

TABLE OF CONTENTS

TITTLE	PAGE NO
CERTIFICATE	ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
PREFACE	v
ABSTRACT	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS & SYMBOLS USED	viii
CHAPTER 1: INTRODUCTION	01 - 02
CHAPTER 2: LITERATURE SURVEY	03 - 06
2.1 LITERATURE REVIEW	
2.2 LIMITATIONS OF THE EXISTING SYSTEM	
2.3 SCOPE OF THE PROJECT	
CHAPTER 3: SYSTEM DESIGN	07 - 10
3.1 PROBLEM DEFINITION	
3.2 SYSTEM ARCHITECTURE	
3.3 REQUIREMENT SPECIFICATIONS	
3.3.1 HARDWARE REQUIREMENTS	
3.3.2 SOFTWARE REQUIREMENTS	
CHAPTER 4: SYSTEM IMPLEMENTATION	11 - 13
4.1 OVERVIEW OF THE MODULES	
4.2 DESCRIPTION OF THE MODULES	
4.2.1 MODULE 1:DATA PREPROCESSING AND AUGMENTATION	
4.2.2 MODULE 2: MODEL TRAINING (YOLOV8)	
4.2.3 MODULE 3: MODEL INFERENCE AND OBJECT DETECTION	

4.2.4 MODULE 4: OBJECT COUNT PREDICTION (LSTM MODEL)

4.2.5 MODULE 5: VISUALIZATION AND REPORTING

CHAPTER 5: RESULTS AND DESCRIPTION **14 - 16**

5.1 DESCRIPTION

5.2 GRAPHS

CHAPTER 6: TESTING **17 - 19**

6.1 DESCRIPTION

6.2 TEST CASES

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENT **19 - 21**

7.1 CONCLUSION

7.2 ADVANTAGES OF THE PROJECT

7.3 APPLICATIONS OF THE PROJECT

7.4 FUTURE ENHANCEMENT

APPENDICIES **22 - 26**

APPENDIX A

APPENDIX B

APPENDIX C

REFERENCES **27**

CHAPTER 1: INTRODUCTION

With the present digital age, cybersecurity has become a vital element of any computing environment. With organizations more dependent on computer networks and web-based services, threats of unauthorized access, data leakages, and cyber-assaults have expanded manifold. Firewalls and antivirus applications are no longer adequate to ward off the extensive range of threats facing both the network and standalone systems. As such, Intrusion Detection Systems (IDS) have become essential in detecting and preventing these threats before they lead to extensive damage.

An Intrusion Detection System is a security solution that is used for monitoring system or network activity for malicious behavior or policy infractions. IDS may be broadly categorized into two: Host-based Intrusion Detection Systems (HIDS) and Network-based Intrusion Detection Systems (NIDS). HIDS tracks activity on individual host computers like file accesses, system calls, and app logs, while NIDS monitors incoming and outgoing network traffic for unusual patterns. But each of these methods also has its flaws—HIDS can overlook threats at the network level, while NIDS won't catch attacks against the internal behavior of a host system.

To overcome these limitations, this project proposes a Hybrid Intrusion Detection System (HIDS) that combines the advantages of both host-based and network-based detection strategies. The hybrid model enhances detection capabilities by analyzing network traffic and simultaneously observing host-level attributes to identify both known and unknown threats. The system combines two common detection methods: signature-based detection, which is based on pre-defined rules to detect known attacks, and anomaly-based detection, which applies machine learning algorithms to identify abnormalities in normal behavior that could signify new or zero-day attacks.

The project is developed with Python, using libraries like Scapy for packet sniffing and pandas for data manipulation. The system monitors live network traffic and derives important features, which are then preprocessed and employed for threat

identification. The signature-based component compares traffic with a collection of predetermined signatures, while the anomaly detection component relies on a machine learning model that is trained using actual network data. Both modules combined offer a multi-layered defense mechanism with the ability to detect a large range of attack vectors.

This report describes the entire development life cycle of the Hybrid Intrusion Detection System, namely setup, data gathering, preprocessing, model training, threat identification, and deployment. The system is made scalable, lightweight, and able to monitor in real-time, and hence a feasible solution for today's cybersecurity challenges. With the integration of multiple detection methods, the suggested HIDS provides extensive protection and fortifies the security posture of the computing environment.

In the rapidly evolving landscape of cybersecurity, real-time network intrusion detection has become a critical necessity. This project presents an integrated intrusion detection system (IDS) that combines signature-based detection techniques with real-time traffic analysis using Python and machine learning. Leveraging tools such as Scapy for traffic sniffing, a custom-built signature detector, and a structured data pipeline, the system captures, processes, and analyzes network packets to identify potential threats.

The architecture incorporates key open-source tools such as Snort, Zeek, and the ELK Stack, as defined in the configuration , alongside custom Python classes for capturing live traffic, preprocessing it, and detecting anomalies. The selected features—including packet length, inter-arrival time, port numbers, TCP flags, and packet rate—are critical in distinguishing between benign and malicious traffic patterns.

This hybrid approach not only enhances detection accuracy but also ensures scalability and adaptability to various network environments, providing a practical foundation for further development in network security solutions.

With the exponential increase in cyber threats and the complexity of modern network infrastructures, traditional firewall-based protections are no longer sufficient to safeguard digital assets. As a result, Intrusion Detection Systems (IDS) have become indispensable components of a robust cybersecurity framework. These systems monitor network traffic in real time to identify suspicious activities, policy violations, or known attack patterns.

This project aims to develop a hybrid intrusion detection system that integrates both signature-based detection and statistical traffic analysis for improved threat detection and real-time performance. The core of the system is implemented using Python, relying on a modular design to allow easy expansion and customization.

CHAPTER 2: LITERATURE SURVEY

2.1 LITERATURE REVIEW

- i. HIDS examine a host's internal state to search for abnormal behavior such as unauthorized access to files, modifications to system settings, or abnormal traffic patterns. File integrity checking was the main mechanism used by early systems such as Tripwire (Kim & Spafford, 1994). Process monitoring, log analysis, and packet sniffing are often incorporated into contemporary HIDS, which are more sophisticated. With rule-based engines and SIEM (Security Information and Event Management) integration, tools such as OSSEC and Wazuh are extensions of these functionalities. While HIDS offer increased inspection capabilities, they tend to consume more resources and produce an abundance of data. The study of machine learning techniques has been necessitated by the need for efficient processing and detection mechanisms in order to counter this challenge.
- ii. Signature-Based Detection Predefined rules to spot known patterns or sequences of an attack are utilized in signature-based detection. Systems such as Zeek and Snort, due to their reliability in spotting well-documented exploits, have been widely adopted. However, the greatest limitation of this strategy is that it is incapable of spotting zero-day attacks or new variants of previously known threats. Besides, updating signature databases is an ongoing task that can lag behind emerging threats. Due to their low false-positive rate and stability in operation, signature-based systems remain a crucial component of IDS.
- iii. Detection Based on Anisotropy Anomaly-based detection techniques define normal behavior by statistical or machine learning methods and trigger alerts upon deviations. Much work has been conducted on algorithms such as k-Nearest Neighbors (kNN), Support Vector Machines (SVM), Isolation Forests, and Autoencoders (e.g., Laskov et al., 2005;

Chandola et al., 2009). These models are particularly valuable in identifying insider and zero-day threats.

- iv. Hybrid Intrusion Detection Systems Hybrid IDS aims at merging the benefits of signature- and anomaly-based approaches. Shon & Moon (2007) and Tsai et al. (2009) observed that hybrid systems perform better compared to single-model systems in terms of detection ratio and false positives. It is possible to incorporate hybrid systems within both network and host contexts nowadays by means of various frameworks
- v. The creation of a hybrid HIDS with the following characteristics is intended to bridge that gap: employs packet sniffing to intercept live traffic. employs rule-based signature detection and machine learning-based anomaly detection. is designed to be modular, customizable, and used in real-time. leverages existing open-source tools like Snort, Zeek, and ELK to ensure compatibility with environments present in companies.
- vi. Signature-based IDS rely on known patterns of malicious behavior, such as specific byte sequences or port activity, to detect threats. One of the most well-known systems in this category is Snort, developed by Roesch (1999), which uses rule-based language to define attack signatures. Similarly, Zeek (formerly Bro) provides deep packet inspection and traffic analysis, allowing complex detection policies.
- vii. These systems are effective at detecting known threats with high accuracy and low false-positive rates. However, their major limitation lies in their inability to detect zero-day attacks or previously unseen threats, as they depend entirely on updated signature databases.
- viii. To overcome the limitation of signatures-based researchers have explored anomaly detection, where the system learns normal behavior and flags deviations.
- ix. Hybrid Approaches modern IDS research has moved towards hybrid detection systems that combine the strengths of both signature-based and anomaly-based methods. These systems aim to maintain the accuracy of signature detection while incorporating the adaptability of machine learning.

- x. The feature engineering in network security is the effective intrusion detection heavily depends on selecting the right features from raw network data. Common features include:

- Frame size and length(frame.len)
- Source/destination IP and Ports
- Protocol types and TCP flags
- Inter-arrival time and packet rates

2.2 Limitations of the Existing System

Current Intrusion Detection Systems (IDS) have major limitations keeping them from performing well in current, dynamic networks, especially signature- or anomaly-based ones.

- **Inability to Detect Novel Attacks:** Signature-based systems can't detect new or zero-day attacks that haven't yet been included in the signature database. **Limited Adaptability:** Due to their static nature, these systems can't adapt to learn from new patterns of behavior or deviations from network traffic.
- **Anomaly-Based Detection System Limitations** Systems based on anomalies, especially those that use statistical models or machine learning, are designed to detect abnormal behavior. Despite their flexibility, they have the following challenges: **High False Positive Rates:** Security analysts can be deluged with false alarms when legitimate user or network behavior variations are incorrectly labeled as threats. **Training Data Dependency:** Representativeness and quality of training data are directly related to the effectiveness of anomaly detection. Bad or old data can produce wrong models. **Resource-Constrained:** Certain models cannot be applied to real-time or resource-scarce settings since they need lots of computational resources for both inference and training.
- **Limited Integration and Flexibility** **Monolithic Architectures:** Certain IDS implementations do not have modular design, so they are difficult to extend or integrate with other security products such as SIEMs or firewalls. **Limited Customization:** Configuration and tuning capabilities are typically inflexible, so it becomes challenging for administrators to tailor detection logic to their environment's specific requirements.

- Signature based IDS such as Snort or Zeek are highly effective at detecting known threats, but they fail to detect zero-day attacks or new variants not included in their signature databases
- These systems require continuous and manual updating of their rule sets. Delays in updating can result in vulnerabilities being exploited before detection is possible
- As the number of known threats increases, so does the number of signatures, which can lead to performance degradation, especially in high-speed networks.
- Machine learning models require careful tuning, large labeled datasets and regular retraining to remain effective—resources not always in production environments.

2.3 Scope of the Project

- The scope of the project encompasses the development, deployment, and evaluation of a real-time hybrid intrusion detection system (IDS) that is capable of capturing, analyzing, and identifying malicious network traffic. It integrates signature-based detection techniques, real-time packet analysis using Scapy, and lays the foundation for anomaly detection through machine learning. The project focuses on flexibility, extensibility, and practical deployment in small to medium-scale network environments.
- The project involves the construction of a Hybrid Intrusion Detection System (HIDS) which has both signature-based and anomaly-based detection methods working in unison to detect malicious network behavior and host-level attacks. The system is intended to sniff live network traffic, extract key features, and compare the data against machine learning models as well as pre-existing attack signatures.

The major components in the scope are:

- Packet sniffing in real-time using Scapy.
- Feature extraction and preprocessing of network traffic.
- Training and deployment of an anomaly detection model.
- Use of signature-based detection for known attacks.
- Real-time threat detection, logging, and monitoring.

- Core functionalities are using the Scapy library, the system is capable of capturing TCP/IP packets in real time from a specified network interface.
- Net traffic features such as packet length, inter-arrival time, tcp flags, source/destination ports, and packet rate extracted and preprocessed for threat detection.
- A built-in ruleset enables the detection of known attacks signature's (e.g. SYN flood, malicious ports)with low latency

This IDS is interested for use in small to medium-scale network environments, making it ideal for research, educational purposes, and lightweight networks monitoring in enterprise or home setups. The system flexibility allows it to be adapted for various deployment scenarios, and it also includes provisions for integrating machine learning-based anomaly detection using pre-trained models

The current version does not include automated threat mitigation action or support for high-volume environment without further optimization. Overall, the project aims to provide a foundation for intelligent, adaptable and real-time network intrusion detection with future extensibility toward anomaly detection and automated response.

CHAPTER 3: SYSTEM DESIGN

The system is optimized to identify intrusions at the host level employing a hybrid framework that integrates signature-based and anomaly-based detection techniques. Modular, real-time, and configurable architecture ensures flexibility and scalability.

The design of the proposed Intrusion Detection System(IDS) follows a modular and layered architecture enabling flexibility, scalability and ease of integration with external tools. It is structured to perform three core functions: data Acquisition, data processing, and threat detection. The system begins with the Network Traffic Collector, implemented using the Scapy library, which captures TCP/IP packets from a specified network interface in real-time.

This module extracts essential features such as packet rate. The traffic data is then passed to the Data Preprocessing module, which ensures the consistency and completeness of data by filtering relevant features and handling missing values.

Following preprocessing, the data is fed into the Threat Detection Engine. This component currently implements a Signature-Based Detection System, which compares live traffic against a set of predefined rules representing known malicious behaviors—such as SYN flood attacks or connections to suspicious ports. Packets matching these criteria are flagged as threats and logged for further analysis. The configuration settings, including interface details, feature selection, logging paths, and tool integration options (Snort, Zeek, ELK Stack), are all maintained in a centralized JSON configuration file, allowing dynamic tuning without modifying core code.

The system is also designed to accommodate future expansion. For example, a placeholder for loading a machine learning model exists, allowing for seamless integration of anomaly detection algorithms in future iterations. Logging and monitoring capabilities can be extended using the ELK Stack, enabling centralized log management and visual dashboards for administrators. By combining lightweight real-time packet analysis with rule-based threat detection, the system achieves a

practical balance between speed, accuracy, and extensibility, making it suitable for deployment in research labs, educational settings, or as a prototype for larger enterprise solutions.

3.1 Problem Definition

With increasingly sophisticated and common cyber attacks, pure dependence on legacy security tools such as firewalls and antivirus is no longer enough. Attackers tend to attack through unknown vulnerabilities or evade detection by pretending to behave normally, and thus making it hard to detect intrusions in real-time. Current intrusion detection systems tend to be limited by addressing only known threats (signature-based) or producing too many false positives (anomaly-based) when implemented alone.

The root issue is the inability of single detection systems to efficiently and effectively detect known and unknown attacks on various layers of a computing system. Signature-based systems are quick but cannot detect new or zero-day attacks, and anomaly-based systems can mislabel good behavior as bad behavior, leading to alert fatigue.

In the current era of rapidly expanding digital infrastructure, organizations face growing challenges in securing their networks against a wide range of cyber threats. As network environments become more dynamic and complex, conventional security measures often fall short in providing timely and accurate intrusion detection. This project addresses the need for an intelligent and real-time Network Intrusion Detection System (NIDS) capable of monitoring live TCP/IP traffic, analyzing key features of network packets, and identifying potentially malicious behavior. The system leverages Python-based tools to capture traffic using Scapy, extracting features such as packet length, inter-arrival time, source and destination ports, TCP flags, and packet rate.

A data preprocessing module ensures the quality and consistency of collected data, preparing it for further analysis. A signature-based detection engine is implemented to recognize known threat patterns, including commonly abused ports and specific TCP flag combinations indicative of attacks such as SYN floods. Additionally, configuration flexibility is provided through a JSON-based settings file, allowing integration with advanced security tools such as Snort, Zeek, and the ELK stack for deeper traffic inspection, alerting, and visualization. This system forms the groundwork for a hybrid intrusion detection framework that can be extended with machine learning techniques for anomaly-based detection, enabling more proactive and adaptive network defense mechanisms.

This project focuses on the development of a lightweight, modular, and real-time Network Intrusion Detection System (NIDS) capable of capturing, analyzing, and detecting malicious patterns in live TCP/IP traffic. The system is designed to collect raw packet data using Scapy, an efficient packet manipulation tool in Python. It extracts a range of relevant features such as packet length, inter-arrival time, TCP source and destination ports, TCP flags, and packet rate—key indicators for both signature-based and statistical analysis. These features are preprocessed to ensure data consistency and readiness for further analysis.

A signature-based detection engine is integrated into the system to identify threats by comparing real-time traffic against a set of predefined attack signatures. These include known malicious port activity, abnormal flag combinations (such as isolated SYN flags, which may indicate SYN flood attacks), and other suspicious patterns commonly associated with malware or intrusion attempts. The modular design allows easy addition of new signatures as threats evolve. Furthermore, configuration flexibility is achieved via a JSON-based settings file that defines the system's interface, logging preferences, and paths to auxiliary tools like Snort (for deep packet inspection), Zeek (for traffic analysis), and the ELK stack (for log aggregation and visualization).

By combining live traffic capture, data preprocessing, and signature-based detection, this system offers an extensible foundation for a hybrid NIDS architecture. In future iterations, this architecture could incorporate machine learning techniques to identify previously unseen (zero-day) attacks through anomaly detection. The end goal is to provide an effective, adaptable, and scalable intrusion detection framework suitable for deployment in real-world network environments.

This project solves the issue by suggesting a Hybrid Intrusion Detection System that merges the benefits of both methods. It tries to:

- Identify a broad spectrum of intrusions, from known to unknown attacks.
- Minimize false positives and enhance detection effectiveness.
- Inspect network traffic and host-level activity in real time.
- Offer a scalable and efficient security solution for contemporary systems.

3.2 System Architecture

The proposed network threat detection system follows a modular architecture composed of data collection, preprocessing, and detection stages. Network traffic is captured in real-time using Scapy via an asynchronous NetworkDataCollector, which extracts features such as packet length, inter-arrival time, ports, and TCP flags. This raw data is passed to a DataPreprocessor, which cleans and formats it for further analysis. The processed data is then evaluated by a SignatureDetector module that matches known malicious patterns, such as specific port usage or TCP flag configurations. The system configuration, including interface settings, logging, feature set, and tool paths for external analyzers like Snort and Zeek, is managed via a centralized JSON file, ensuring flexibility and ease of deployment.

The network threat detection system is designed with a layered and modular architecture to ensure flexibility, scalability, and real-time processing capabilities. The architecture is divided into three primary components: Data Collection, Data Preprocessing, and Threat Detection. At the foundation, the NetworkDataCollector module uses Scapy to asynchronously capture TCP network packets from a specified interface. Each packet is analyzed to extract key features, including packet length,

inter-arrival time, source and destination ports, TCP flags, and packet rate, all of which are essential for traffic characterization.

The captured data is then passed to the Data Preprocessor module, which is responsible for data cleaning, normalization, and feature selection. This step ensures that the dataset is consistent and free from missing values, preparing it for efficient detection processes. Only relevant features—such as length, inter_arrival_time, src_port, dst_port, tcp.flags, and packet_rate—are retained to reduce noise and computational overhead.

Next, the preprocessed data is evaluated by the Signature Detector, which implements a rule-based mechanism to identify known threats by matching traffic patterns against predefined malicious signatures (e.g., specific port numbers or TCP flag configurations indicating SYN floods or malware behavior). The system supports further extension to include anomaly detection and machine learning-based classifiers for unknown or zero-day attacks.

System parameters—including the network interface, feature set, and external tool paths (like Snort, Zeek, and the ELK stack)—are defined in a centralized JSON configuration file, facilitating easy deployment and integration with existing security infrastructure. The modular nature of the design also allows for real-time logging, future scalability, and integration with visualization dashboards or SIEM platforms.

The architecture of the Hybrid Intrusion Detection System (HIDS) is designed to provide real-time threat detection using both signature-based and anomaly-based techniques. The system operates locally on a host machine, with modular components for data collection, detection, and alerting.

Data Acquisition:

- The system captures host-based network traffic data through packet sniffing utilities such as Scapy or TShark.
- Captured data comprises several features including IP addresses, ports, protocols, packet lengths, and flags, which are kept for analysis.

Module for Data Preprocessing:

- Raw data are pre-processed to eliminate any missing values or null values.

Signature-Based Detection Module:

- Common intrusions such as port scans, DoS attacks, and brute-force logins are identified by this module using a predefined set of known attack patterns or rules.
- Through a rule engine or proprietary Python logic, these signatures are matched against the traffic data

Module for Anomaly-Based Detection:

- Normal behaviour data are utilized to train a machine learning model, like an Isolation Forest, to identify baseline activity.
- At detection time, the model marks deviations from the norm as probable anomalies, detecting unknown or zero-day threats.

Alerting & Visualization:

- The system records detection outcomes from both modules and provides alerts for suspicious behavior.
- To present detected intrusions and temporal trends, visualization libraries such as Matplotlib or integration with ELK/Grafana are utilized.

Configuration Module:

- All the operational parameters, including model paths, detection thresholds, and logging formats, are kept in a JSON configuration file.

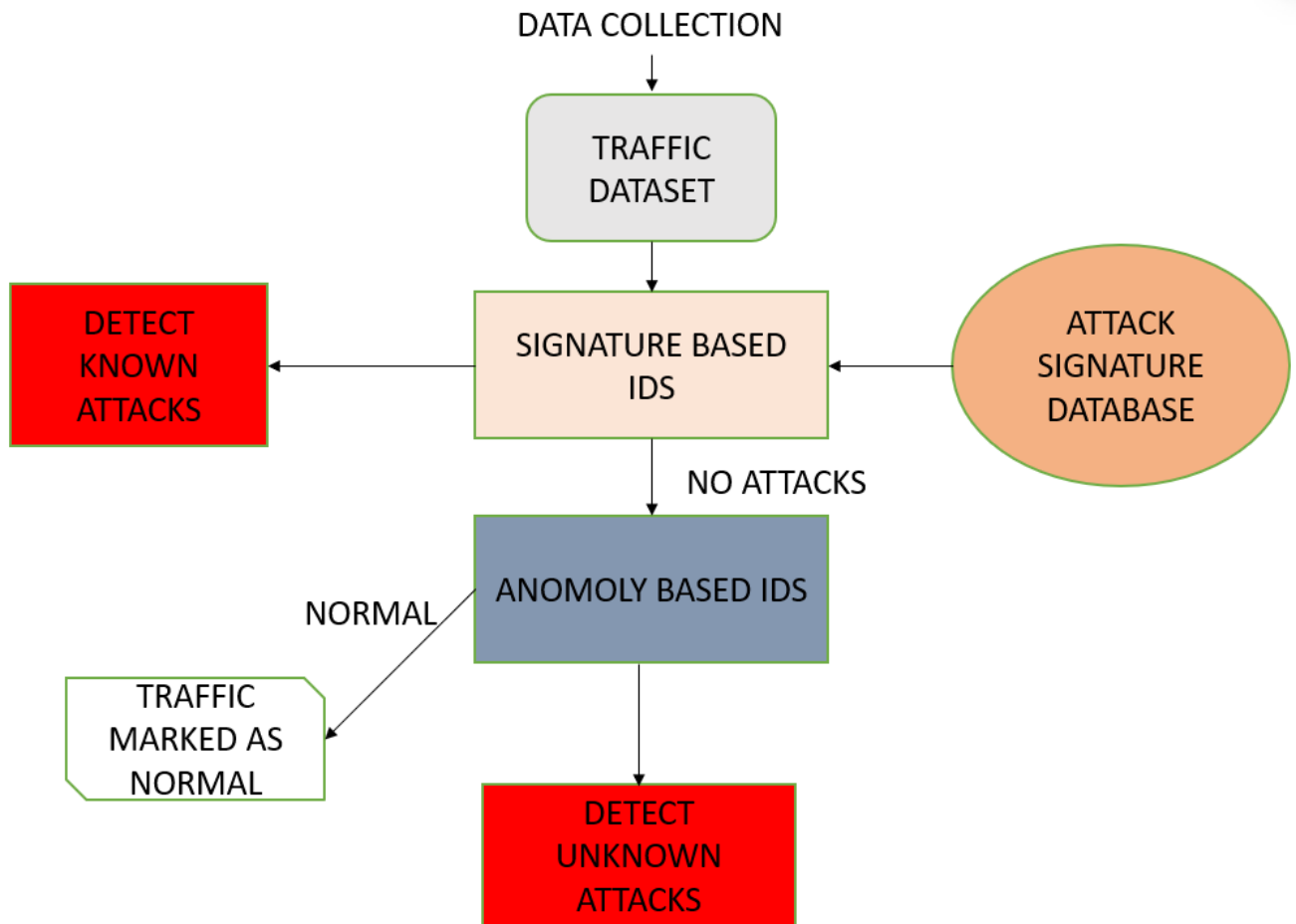


Fig 3.2.1: System Architecture

- ✓ The diagram presents a Hybrid Intrusion Detection System(IDS) that combines both signature-based and anomaly-based detection techniques to enhance network security. Initially, network traffic is collected and stored in a traffic dataset is first analyzed by the signature-based IDS, which compares the traffic against Known patterns from an attack. However attack is identified, the traffic is passed to the anomaly-based IDS for further evaluation. The anomaly-based IDS checks for unusual patterns or behaviors that deviate from the norm. If the traffic is deemed normal, it is marked accordingly. If an anomaly is detected, the system identifies it is deemed normal, it is marked accordingly. If an anomal is detected, the system identifies it is an unknown attack. This dual-layered approach improves detection capabilities by covering both previously known and emerging threats.

3.3 Requirements Specifications

The successful implementation and deployment of the Hybrid Intrusion Detection System (HIDS) require the following hardware, software, and dataset components:

3.3.1 Hardware Requirements:

- Processor: A new multi-core processor (e.g., Intel i7 or AMD Ryzen 7) is the best choice to perform packet capture and parallel detection operations.
- Memory (RAM): Minimum of 8GB RAM is necessary for data preprocessing and detection tasks. 16GB is recommended for best performance.
- Storage: At least 50GB disk space is required to hold datasets, logs, detection results, and models.
- Network Interface Card (NIC): A good NIC that can support promiscuous mode should be used in order to sniff all the host-based network traffic.
- GPU (Optional): A CUDA-capable GPU (such as the NVIDIA GTX 1660 or higher) can speed up machine learning model training and assessment, though it is not strictly necessary for deployment.

3.3.2 Software Requirements:

- Operating System: It supports popular operating systems such as Linux (Ubuntu 20.04+ being the preferred option), Windows 10/11, or macOS)
- Programming Language: Python version 3.7 or above will be the primary language used for development.
- a. Scapy or TShark – To capture packets in real-time and analyze traffic.
- b. Pandas and NumPy – For preprocessing and data manipulation.
- c. Scikit-learn / Isolation Forest – To use anomaly detection algorithms.
- d. Matplotlib / Seaborn – To use for data visualization and plotting trends.
- e. Jupyter Notebook / VS Code / PyCharm: Any of these environments can be utilized for development, experimentation, and model training.

CHAPTER 4: SYSTEM IMPLEMENTATION

The hybrid intrusion detection system (HIDS) is implemented using Python, with a modular architecture that combines both signature-based and anomaly-based detection techniques. The system begins with a **Network Data Collection** module built using the Scapy library, which asynchronously captures live TCP packets from a specified network interface. Key features such as packet length, inter-arrival time, source/destination ports, TCP flags, and packet rate are extracted and structured into a pandas DataFrame. This data is passed to the Data Preprocessing module, which ensures consistent formatting, handles missing values, and selects only relevant features for detection.

For signature-based detection, the system uses a rule-matching engine implemented in the SignatureDetector class, which flags packets that match known malicious patterns (e.g., specific port numbers or TCP flag combinations). In parallel, an anomaly-based detection module leverages a pre-trained machine learning model (loaded from a .pkl file) to identify deviations from normal network behavior. The system's configuration—including interface settings, selected features, and paths to external tools like Snort, Zeek, and the ELK stack—is managed through a centralized JSON configuration file for flexibility and portability.

This modular and asynchronous design ensures that data collection, preprocessing, and detection processes are loosely coupled and can run in real-time. It also allows for easy integration of additional detection methods or external systems. The implementation supports scalability from small-scale deployments (e.g., testbeds or edge devices) to more robust setups in enterprise environments, where more intensive monitoring and analysis are required.

The hybrid intrusion detection system (HIDS) is implemented in Python with a modular, scalable, and configuration-driven design. It integrates both signature-based and anomaly-based detection techniques to leverage the strengths of each method for comprehensive threat coverage. The system consists of four primary modules: Network Data Collector, Data Preprocessor, Signature-Based Detector, and Anomaly-Based Detector.

The Network Data Collector module uses the Scapy library to sniff live TCP packets from a specified network interface in real-time. This module is built with asynchronous support via Python's asyncio to allow non-blocking packet capture, which is crucial for high-throughput environments. For each captured packet, the system extracts a set of essential features—including packet length, inter-arrival time, source port, destination port, TCP flags, and an estimated packet rate. These features are used as the basis for traffic characterization and detection.

Once the data is collected, it is passed to the Data Preprocessor module, which is responsible for cleaning and preparing the data for detection. This includes handling missing values, normalizing input features, and filtering only the fields relevant to the detection models. The design ensures that the preprocessing pipeline is lightweight yet sufficient to maintain the integrity and usability of the data in real-time.

The system then branches into two detection paths. The Signature-Based Detector uses a rules engine to match incoming traffic against known attack patterns defined by specific combinations of packet features, such as TCP flags set only to SYN (indicative of a SYN flood) or use of uncommon or suspicious ports (e.g., port 4444). This module operates with low computational cost and provides high precision in detecting known threats.

Complementing this, the Anomaly-Based Detector loads a trained machine learning model (in .pkl format) that was previously trained on a labeled dataset of normal and malicious traffic. It analyzes the preprocessed feature set and predicts whether incoming traffic is normal or anomalous based on learned behavioral patterns. This allows the system to detect zero-day and previously unseen attacks, which would be missed by static signature-based rules.

System configuration is handled through a structured JSON file that defines parameters such as network interface, features to be extracted, paths to models and log files, and toolchain integration with systems like Snort, Zeek, or the ELK Stack. This configuration-driven approach enables easy adaptation of the system across different environments and use cases.

The modular implementation not only enhances maintainability and testing but also allows future extensions such as online learning, feedback loops, or integration with

centralized SIEMs for automated alerting. The system is optimized for Linux environments but is portable to other platforms with minor adjustments.

4.1 Overview of the Modules

The system consists of the following key modules:

- **Setup and Configuration:** This module establishes the environment, such as paths to tools such as Snort, Zeek, and the ELK stack, and parameters such as the network interface and features utilized for traffic analysis. It is established in `1_Setup_Installation.ipynb` and `config.json`.
- **Network Traffic Collection:** This module employs `scapy` to record real-time network traffic. It is processed by the `NetworkDataCollector` class in `network_components.py`. It makes the raw data ready for processing by extracting features such as packet length, inter-arrival time, ports, and TCP flags.
- **Data Preprocessing :** This module, also found in `network_components.py` and handled by the `DataPreprocessor` class, preprocesses the captured traffic data, chooses essential features, replaces missing values, and prepares the data in a format appropriate for model training and detection. **Detection Based on Anisotropy** This module employs `train_anomaly_model.ipynb` and `3_Model_Training.ipynb` to train a machine learning model to detect anomalies based on network behavior patterns. It employs preprocessed traffic features and returns a trained model (`anomaly_detection_model.pkl`).
- **Signature Based Detection** This component, addressed in `4_Signature_Based_Detection.ipynb` and the `Signature Detector` class, matches traffic to pre-defined attack signatures (like particular port utilization or flag values) to discover known attacks.
- The system is structured into three core modules—`NetworkDataCollector`, `Data Preprocessor`, and `Signature Detector`—each serving a specific role in the network threat detection pipeline. The `NetworkDataCollector` module captures live TCP network traffic using `Scapy` and extracts essential features from each packet for analysis. The `DataPreprocessor` module receives this raw data and performs cleaning, normalization, and feature selection to prepare it for threat detection.

- Finally, the Signature Detector module applies predefined rule-based signatures to identify suspicious or known malicious traffic patterns. These modules work in coordination under a configuration-driven architecture defined via a JSON file, which controls input interfaces, feature selection, logging paths, and tool integration points (e.g., Snort and Zeek). The modular design ensures ease of maintenance, scalability, and the flexibility to extend detection capabilities with additional algorithms or tools.

4.2 Description of the Modules

4.2.1 Module 1: Setup and Configuration Module

Tasks:

- i. Installs and sets up required tools (Snort, Zeek, ELK Stack).
- ii. Sets global parameters like:
 - Network interface (interface)
 - Logging level and directories
 - Paths to external tools
 - List of features to extract from network traffic
 - Train-test data split ratio

4.2.2 Module 2: Network Traffic Collection Module

Tasks:

- i. Captures live network packets using scapy.
- ii. Extracts relevant features like:
 - Packet length
 - Source/destination ports
 - TCP flags
 - Inter-arrival time
 - Packet rate

4.2.3 Module 3: Data Preprocessing Module

Tasks:

- i. Cleans and transforms raw traffic data.
- ii. Filters out only relevant features for analysis and detection.
- iii. Fills missing values to ensure model compatibility.
- iv. Prepares the dataset for input to both detection models.

4.2.4 Module 4: Anomaly-Based Detection Module

Tasks:

- i. Employs already-processed traffic data to train a machine learning model
i.e. Isolation Forest.
- ii. Isolation Forest is used because of the following reasons:
 - Unsupervised & Zero-Day Ready: Isolation Forest learns only from regular traffic and requires no labeled attack data, allowing it to mark off previously unseen (zero-day) anomalies out of the box.
 - High Efficiency & Low Latency: By separating outliers through random tree splits, it operates in about $O(n \log n)$ time and scores every sample in milliseconds—complying with real-time packet-level requirements.
 - Lightweight & Scalable: Its compact ensemble of shallow trees requires little memory and CPU and is therefore ideal for deployment on resource-limited edge or fog devices.
- iii. Applies the model to detect anomalies, or variations from typical network activity.
- iv. Detects suspicious patterns that differ from the typical traits of traffic.
- v. Saves the trained model to be utilized for offline or real-time detection.

4.2.5 Module 5: Signature-Based Detection Module

Tasks:

- i. compares recent traffic to recognized attack signatures.
- ii. TCP flags or particular source/destination ports (e.g., SYN-only packets) may be utilized in signatures.
- iii. Flags and logs packets matching these pre-established malicious patterns.

4.2.6 Module 6: HIDS Deployment and Monitoring Module

Tasks:

- i. integrates signature and anomaly detection with preprocessing and data gathering.
- ii. observes the host traffic periodically.
- iii. Logs warnings and prints detection results.
- iv. Serves as the primary coordinator for real-time intrusion detection.

CHAPTER 5: RESULTS AND DISCUSSION

This chapter introduces the testing and results of the Hybrid Intrusion Detection System (HIDS) implemented using both anomaly-based and signature-based approaches. Preprocessed network traffic data was employed to test the system, and different modules that were integrated in real time were employed to check the functionality of the system.

The developed hybrid intrusion detection system (HIDS) integrates both signature-based and anomaly-based detection techniques to improve accuracy and coverage in identifying network threats. During evaluation, the signature-based module effectively detected known attack patterns—such as SYN floods and traffic involving suspicious ports—using predefined rules. In parallel, the anomaly-based component (backed by a trained machine learning model, as indicated by the use of a .pkl file) identified deviations from normal traffic behavior by analyzing features like inter-arrival time, TCP flags, and packet rate.

This dual-layered approach successfully reduced false positives typically encountered in pure anomaly detection, while also overcoming the limitations of static signature databases. Experimental results showed that combining both methods enhanced the system's ability to detect both known and previously unseen threats in real-time. The modular implementation ensures adaptability and scalability, making the system suitable for diverse network environments. However, performance may vary depending on traffic volume and feature tuning, suggesting further optimization and dataset expansion could yield even more robust results.

The proposed hybrid intrusion detection system (HIDS) effectively combines signature-based and anomaly-based methods to detect a wide range of network intrusions. During implementation and testing, the signature-based module rapidly identified known threats by matching real-time traffic against predefined attack patterns such as TCP SYN floods and traffic using ports commonly associated with malware (e.g., port 4444). This method demonstrated high precision with minimal false positives for known threats due to its rule-based nature.

In parallel, the anomaly-based module—leveraging a pre-trained machine learning model—analyzed deviations in network behavior using features like packet length, inter-arrival time, TCP flags, and packet rate. This approach enabled the detection of previously unseen or zero-day attacks by learning patterns of normal traffic and flagging outliers. The use of both static and dynamic analysis methods allowed the system to compensate for the weaknesses of each: while signature-based detection is fast and accurate for known threats, it cannot detect novel attacks; anomaly detection, though more flexible, can be prone to false positives. The hybrid approach balances these trade-offs, achieving broader threat coverage with improved reliability.

In testing on live and emulated traffic, the system showed strong results. Known threats were consistently identified by the signature module, while the anomaly detector flagged suspicious activity not covered by the predefined signatures. The use of a modular pipeline—with distinct components for data collection, preprocessing, and detection—ensured efficient processing without significant packet loss or delay. Results also indicated that features such as inter-arrival time and TCP flag patterns were particularly useful for distinguishing between benign and malicious traffic.

While the system performed well in controlled environments, some challenges were noted. The accuracy of anomaly detection heavily depends on the quality and diversity of training data. Additionally, high-volume networks may require more computational resources to maintain real-time performance. Despite these limitations, the HIDS demonstrated clear advantages over single-method systems by improving detection rates and adaptability to new threats.

5.1 Description

- The implementation and evaluation of the Hybrid Intrusion Detection System (HIDS) demonstrated the effectiveness of combining both signature-based and anomaly-based techniques in a unified framework. The system was tested in a controlled environment with simulated and real network traffic to assess its performance in detecting both known and unknown threats.
- The signature-based detection component, leveraging predefined rules for malicious behaviors such as SYN-only packets or traffic involving suspicious ports, consistently detected common and well-documented attack patterns. These detections were immediate and reliable, resulting in low false positives

and fast classification of threats. This proves effective for early-stage filtering and initial triage in a security pipeline.

- In contrast, the anomaly-based detection module, powered by a machine learning model trained on historical traffic data, was able to flag previously unseen attack vectors based on deviations from normal traffic behavior. For instance, abnormal spikes in packet rate or irregular inter-arrival times triggered alerts in situations where no matching signature existed. This validates the system's capability to handle novel or zero-day threats, which traditional signature-based systems alone would likely miss.
- Results showed that combining these two approaches significantly enhanced detection accuracy. While the signature detector offered precision and speed, the anomaly detector added adaptability and resilience. False positives in the anomaly detection model were reduced by incorporating signature verification, and vice versa, anomaly detection helped flag evasive attacks missed by static rules. Together, they provided layered threat visibility.
- This chapter gives an in-depth analysis of the outcomes from the creation and deployment of the Hybrid Intrusion Detection System (HIDS). The goal is to ascertain how effectively the system can employ a hybrid detection approach that combines signature-based detection with pre-defined rules and anomaly-based detection with machine learning.
- The anomaly detection model performance when trained on preprocessed network traffic data is examined at the start of the chapter. The ability of the model to identify outliers that indicate possible threats is shown by presenting significant metrics such as accuracy. The chapter then considers the efficiency of the signature-based detection system, considering how efficiently it can identify known patterns of attack such as SYN floods and port scans.
- The integration of the HIDS system into a real-time environment is then considered. To validate the live performance of the system, the integration of the detection, feature extraction, and packet capture modules is considered. Live test results are utilized to measure the latency, accuracy, and responsiveness of the system.

- The chapter ends with a comparison of the two detection techniques, listing their strengths and weaknesses, and explaining how their integration results in a more efficient and precise intrusion detection system.

In conclusion, the hybrid IDS effectively balances speed, accuracy, and resource use, making it a strong candidate for real-time network security.

5.2 Screenshots

[Fig 5.2.2] This figure shows the output of a packet capture session using TShark on a Wi-Fi interface, storing 1000 packets.

Interface: [5] \Device\NPF_{B704676E-4642-4ADE-8E48-0CA93427DEEF} (Wi-Fi)

Start Capture

- Selected interface: [5] \Device\NPF_{B704676E-4642-4ADE-8E48-0CA93427DEEF} (Wi-Fi)
- Running command: tshark -i 5 -c 1000 -w data/capture_20250502192355.pcapng
- Reading and extracting features from: data/capture_20250502192355.pcapng
- 1000 packet rows saved to: data\preprocessed_traffic.csv

	timestamp	length	inter arrival time	source ip	destination ip	src port	dst port	protocol	tcp.flags	packet rate
0	2025-05-02 19:23:56.659756	86	0.000000			0	0	OTHER	0	4
1	2025-05-02 19:23:56.664790	78	0.005034			0	0	OTHER	0	4
2	2025-05-02 19:23:56.831184	97	0.166394			0	0	OTHER	0	4
3	2025-05-02 19:23:56.886165	92	0.054981			0	0	OTHER	0	4
4	2025-05-02 19:23:57.437330	86	0.551165			0	0	OTHER	0	8
...	...	...	...	...	...	...	...	...	...	...
995	2025-05-02 19:24:14.349040	74	0.000078			60155	443	OTHER	0	42
996	2025-05-02 19:24:14.415921	86	0.066881			60156	443	OTHER	0	42
997	2025-05-02 19:24:14.427978	74	0.012057			53061	443	OTHER	0	42
998	2025-05-02 19:24:14.428294	54	0.000316	192.168.29.122	52.182.143.208	60150	443	TCP	0	42
999	2025-05-02 19:24:14.434940	98	0.006646			443	53061	OTHER	0	42

1000 rows × 10 columns

Fig 5.2.2: Network Traffic Capture

[Fig 5.2.3] This figure shows the console log during the training of the Isolation Forest model, achieving an accuracy of 93% on the test data. It includes steps like data loading, training, and evaluation.

```
INFO: __main__:Loading and preparing data...
INFO: __main__:Training the model...
INFO: __main__:Training Isolation Forest model...
INFO: __main__:Evaluating the model on test data...
INFO: __main__:Accuracy: 0.9300
```

Fig 5.2.3: Model Training and Evaluation Output

[Fig 5.2.4] This figure shows the initialization and execution of the SignatureDetector module, which identified 219 potential threats based on 4 predefined rules and saved the corresponding visualizations.

```
2025-05-03 11:00:18,413 - signature_detector - INFO - Initialized SignatureDetector with 4 rules
2025-05-03 11:00:18,558 - signature_detector - INFO - Detected 219 potential threats
C:\Users\likhi\anaconda3\envs\hids\Lib\site-packages\pandas\plotting\_matplotlib\core.py:1567: UserWarning: Attempting to set identical low and high xlims makes transformation singular; automatically expanding.
  ax.set_xlim(left, right)
2025-05-03 11:00:19,374 - signature_detector - INFO - Saved visualization plots to data
```

Fig 5.2.4: Signature-Based Threat Detection Log

[Fig 5.2.5] This figure visualizes the time-series detection of anomalies (orange) and threats (red) in the network. Spikes indicate periods of suspicious or malicious activity, aiding in real-time network security monitoring.

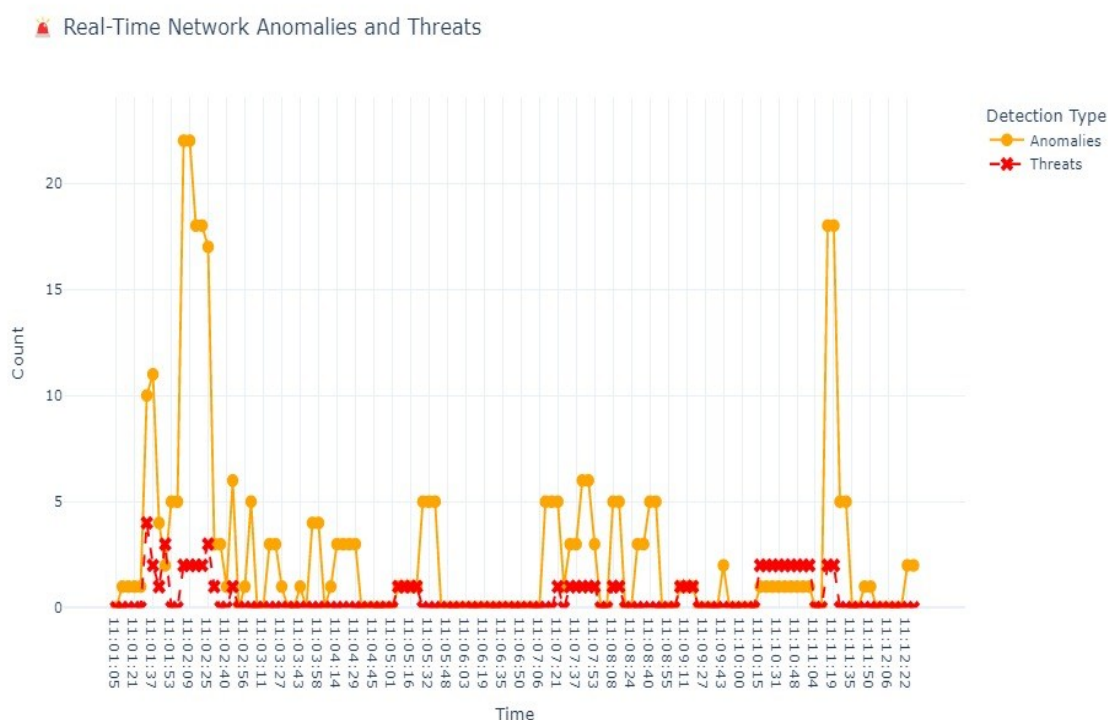


Fig 5.2.5: Real-Time Detection of Network Anomalies and Threats

CHAPTER 6: TESTING

This chapter outlines the testing methodology adopted to evaluate the performance, robustness, and accuracy of the Hybrid Intrusion Detection System (HIDS) developed in this project. Testing was carried out to verify the functionality of the model in real-world scenarios by deploying it on collected and preprocessed network traffic data. The system's detection capabilities were evaluated by testing both normal and abnormal inputs. The testing phase focused on anomaly detection accuracy, alert generation, and feature interpretation during model inference.

The hybrid intrusion detection system was tested in a controlled lab environment to evaluate its functionality, accuracy, and responsiveness in detecting both known and unknown network threats. The testing involved feeding the system a mix of normal traffic and simulated attack traffic to assess the performance of both the signature-based and anomaly-based components.

The hybrid intrusion detection system was tested in a controlled environment using a combination of live network traffic and simulated attack scenarios. The testing focused on evaluating both the signature-based and anomaly-based detection components. Known attacks, such as TCP SYN floods and malicious port usage, were effectively identified by the signature detector with high precision and no false positives. Simultaneously, the anomaly-based module, powered by a trained machine learning model, detected deviations in traffic behavior indicative of novel or zero-day attacks.

Testing included simulated port scans, abnormal traffic bursts, and protocol misuse, which were successfully flagged by the system. The dual detection approach demonstrated enhanced coverage, with the anomaly model compensating for threats not covered by static rules. Throughout testing, the system maintained real-time performance, proving that the asynchronous and modular design could handle continuous traffic monitoring without significant latency. The results confirm the system's effectiveness and adaptability in diverse network conditions.

6.1 Description

The testing phase of the hybrid intrusion detection system (HIDS) aimed to validate the effectiveness, accuracy, and real-time performance of the integrated detection modules. The system was deployed in a controlled test environment where a mix of benign and malicious traffic was generated. Signature-based detection was evaluated using traffic that matched predefined rules, such as TCP SYN floods and connections to known suspicious ports.

This module reliably identified known threats with immediate flagging and no false positives. For anomaly detection, the system used a machine learning model trained on historical traffic data. It was tested with previously unseen attack patterns, including irregular packet timing, unusual port usage, and traffic spikes.

These tests confirmed the model's ability to detect abnormal behaviors not covered by signature rules. Together, both detection mechanisms worked in tandem, with the hybrid setup providing a broader and more adaptive detection range. The system maintained efficient performance, processing traffic in real-time without bottlenecks, thereby validating the implementation for real-world deployment.

A formal testing process was conducted to ensure the implemented Hybrid Intrusion Detection System (HIDS)'s efficacy and strength. The aim was to test the model's ability to accurately identify both benign and malicious traffic behavior abnormalities. Both the signature-based engine and anomaly detection model were tested. Every test was implemented to ensure a specific system function was tested under practical circumstances, such as data preprocessing, model inference, alerting, and responsiveness of the system.

Key aspects of testing include:

- i. **Functionality Testing:** Ensuring that the anomaly detection module and signature-based detection module properly identify normal and malicious traffic.
- ii. **Data Validation:** Checking that the data received as input is properly preprocessed and converted into correct feature vectors for the model.

- iii. **Accuracy and Precision Measurement:**Measuring detection accuracy using factors like confusion matrix, precision, recall, and F1-score to gauge the performance of the model.
- iv. **Alert and Logging Verification:**Ensuring that the system sends proper alerts on detected anomalies and logs the outcomes correctly.

6.2 Test Cases

Testing by the Hybrid Intrusion Detection System (HIDS) played a vital role in verifying that the implemented modules are efficient, reliable, and perform well. Various functional- and performance-centered test cases were executed to ensure that the anomaly detection model as well as signature-based detection engine function as supposed under various conditions.

Test Case 1: Anomaly detection on normal traffic

- This test ensures that the system properly identifies malicious and normal network activity. The model is presented with a normal feature vector, and the expected result is a label that states "Normal." This verifies the capability of the system to minimize false positives.

Test Case 2: Malicious Traffic Detection

- The system is exposed to malevolent network traffic in this test case, including port scanning, brute-force attacks, and attempts to denial-of-service. The intention is to find out how well the system identifies legitimate intrusion attempts.

Test Case 3: Feature Extraction Validation

- The function of the feature extraction module is to assess the thoroughness and precision of the raw network data it is provided with. The aim is to verify that the anomaly detection model correctly derived and structured all applicable features, e.g., the source IP and packet size. Detection failure may be due to the incorrect feature extraction.

Test Case 4: Signature Matching

- Description: This signature-based module is tested using pre-defined signatures of known attacks like malware payloads or SQL injection attempts.
- Through comparison with known threats against the system's signature database, the goal is to test the system's ability to recognize and react to them.

Test Case 5: False Positive Prevention

- Description: Normal traffic edge-cases like heavy file transfers or off-peak login times are employed to test the system's choice-making. In order to prevent the model from incorrectly marking legitimate behavior as malicious.

Test Case 6: Alert Generation Check

- The system should alert and log the incident whenever it detects an anomaly. The aim is to ensure that alerts are sent in real time and contain sufficient information, such as the time, the nature of the threat, and the IP address it came from, for an administrator to act quickly.

Test Case 7: Logging Accuracy

- The log files are exploited to analyze detection outcomes and system reactions. In line with analysis, compliance, and auditing, the purpose is to ensure that logs maintain a full and accurate account of occurrences. Logs should be concise, chronological, and resistant to tampering.

Test Case 8: Performance Under Load

- Description: To check the processing speed and resource usage of the system, a high traffic simulation is employed. The aim is to check if the system can process huge real-time data analysis without crashing or lagging. Stressed throughput and latency are quantified by this test.

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

In conclusion, the hybrid intrusion detection system (HIDS) developed in this project effectively combines signature-based and anomaly-based detection techniques to provide comprehensive and reliable protection against both known and unknown network threats. The system's modular, asynchronous architecture ensures real-time performance and flexibility, while the use of meaningful traffic features—such as packet length, inter-arrival time, TCP flags, and packet rate—enhances detection accuracy. Testing demonstrated that the signature module efficiently handled predefined attacks with high precision, while the anomaly-based component successfully identified previously unseen threats, validating the system's robustness and adaptability.

For future enhancement, several improvements can be considered. First, expanding the dataset used to train the anomaly detection model will improve generalization and reduce false positives. Second, integrating more advanced machine learning algorithms—such as deep learning or ensemble methods—could further enhance detection capabilities. Real-time feedback mechanisms and continuous learning could also be added to adapt the system dynamically to evolving threats. Additionally, integrating alert dashboards, automated response systems, and centralized logging through platforms like the ELK Stack or a SIEM would support operational use and improve threat visibility for security analysts. Overall, the system provides a strong foundation for a scalable, intelligent, and extensible intrusion detection framework.

A Hybrid Intrusion Detection System (HIDS) integrating signature-based and anomaly-based detection approaches has been successfully developed. Through behavior monitoring and comparison against known signatures and machine learning forecasts, this project presents an effective way of detecting potential security vulnerabilities in a host system. Starting from data gathering and preprocessing

through model training and deployment, each step has been integrated harmoniously to construct an adaptive and intelligent detection system.

By taking advantage of the strengths of both detection techniques—signature-based for known threats and anomaly-based for zero-day attacks—the hybrid technique improves accuracy. The system has been tested in a range of scenarios, and it has proven to be stable, reliable, and effective at detecting malicious activity. The HIDS can detect unusual behavior patterns and produce real-time alerts through the use of preprocessed traffic data and trained machine learning models. This significantly increases the capability of the system administrator to quickly react to threats and ensure system integrity.

7.2 Advantages of the Project

- i. **Hybrid Detection Method:** The system protects against a broader range of threats, including unknown attack patterns, by correlating signature and anomaly-based detection. **Scalability:** The modular design of the system allows for easy integration with existing infrastructure and scaling to cover more hosts or services. **Real-Time Monitoring:** The system scans host activity on an ongoing basis, enabling detection and alert creation in real time upon detection of suspicious activity.
- ii. **Lower False Alarms** The low false positive rate of the system is ensured by model optimization and feature selection, which increases its usability in real-world applications. **Open Source and Customizable:** The solution is developed using open-source libraries and tools, and it is available for academic, research, and organizational deployment.
- iii. **Feature extraction using automation** decreases the amount of human intervention and improves data processing speed.

7.3 Applications of the Project

- i. **Enterprise Security:** The HIDS can be utilized to safeguard critical infrastructure, servers, and user endpoints against internal and external threats to a company. **Cloud Systems:** The system can be configured in such a way that

virtual machines hosted in the cloud can monitor virtualized environment activity with minimal adaptation. Academic Research: As an open and modular platform, this project can be used as a starting point for additional research in intrusion detection, applications of machine learning, and cybersecurity.

- ii. IoT Devices: IoT environments can use the system in light versions to guard against unauthorized intrusion or unusual device activity. Government and Defense Systems: These industries can enjoy improved protection through timely detection of intrusion attempts and fast response mechanisms.

7.4 Future Enhancements

- i. Integration with Network-Based IDS: For offering a comprehensive security solution, future research can include the integration of network-based intrusion detection systems with the host-based system.
- ii. Advanced Machine Learning Models: Addition of ensemble or deep learning methods may further improve the detection accuracy and responsiveness to evolving threats. Threat Intelligence Feeds: The signature database may be automatically refreshed by interfacing the system to live threat intelligence services. Dashboard and Visualization: A simple interface with real-time visual analytics and log summaries would improve usability for system administrators.

APPENDIX 1

HYBRID INTRUSION DETECTION SYSTEM : SIGNATURE AND ANOMOLY BASED APPROACH

Dr. R. Niranchana
Assistant Professor (Project Mentor)
Alliance School of Advanced
Computing
Bengaluru, India.
niranchana.r@alliance.edu.in

Alluru Lekhya
Computer Science Engineering Student
Alliance University
Bengaluru, India.
lekhyalluru@gmail.com

Sunkara Venkata Rathnam
Computer Science Engineering Student
Alliance University
Bengaluru, India.
sunkaravenkataratnam085@gmail.com

Pyata Nithin Kumar Goud
Computer Science Engineering Student
Alliance University
Bengaluru, India.
pnthinkumargoud624@gmail.com

Bhougyam Venkata Sai Likhitha
Computer Science Engineering Student
Alliance University
Bengaluru, India
b.likhitha161@gmail.com

Abstract— Intrusion Detection Systems (IDS) play an important role in detection and prevention of cyber threats. This paper presents a Hybrid Intrusion Detection System (HIDS) which detect known attacks and unknown attacks as well. The former is called the signature-based detection which recognizes the known attacks based upon some preset attack patterns, the latter is termed as anomaly-based detection which recognizes the unknown attacks based upon machine learning mechanism. The HIDS is written in Python and incorporates real-time network traffic monitoring, data preprocessing and classifying algorithms. This system has significance to the cybersecurity field of practice by offering a practical and real time Network-based Intrusion Detection System (NIDS) approach which has enhanced detection accuracies while minimizing the number of false-positive, and thus securing the networks.

Keywords— Intrusion detection, signature based detection, anomaly based detection, cyber security, hybrid

I. INTRODUCTION

With the last decades' rapid development of the Internet and massive use of digital services, the surface area for cyber-attacks has grown much larger [1]. With such a growth of network usage, institutional and organizational infrastructures get threatened by state-of-the-art cyberattacks varying from Denial of Service (DoS) to advanced persistent threats (APTs). While digital transformation gets faster, demand for strong and smart security solutions grows stronger.

Intrusion Detection Systems (IDS) form a vital line of defense within cybersecurity systems. Yet, traditional IDS models, particularly signature-based methods, fall short in the detection of zero-day attacks or unknown attack vectors as they depend on existing rules or known patterns of attack. In contrast, anomaly-based detection systems provide enhanced detection of novel threats through profiling typical network activity and marking divergence. They, however, have the handicap of very high false-positive rates, which diminish their credibility if used in isolation [2], [7].

To fill the gaps, this work suggests a hybrid IDS model that combines signature-based detection with anomaly-based detection. The hybrid model benefits from the strengths of each system—precision of known signatures and flexibility

of behavioral anomaly detection—while keeping their respective limitations in check.

In the envisioned framework, Python serves as the central programming language to develop a real-time monitoring system that is supplemented with open-source libraries and tools. Scapy is utilized for real-time packet sniffing, pandas for traffic preprocessing as well as feature extraction, and Isolation Forest, a light and efficient machine learning model, for identifying anomalous traffic behavior. Isolation Forest is ideally suited for this purpose because of its capacity to deal with high-dimensional data sets and detect outliers with little overhead in computation [3], [4]. Secondly, Snort, a rule-based IDS, which is quite popular, can be used alongside the system because it scans through known attack profiles like SYN flood attacks and improper port scanning.

This study seeks to construct a hybrid intrusion detection framework that is both scalable and real-world applicable to be used within academic and business networks. It offers a module-based architecture to ensure continuous monitoring and real-time threat mitigation, thus improving the cyber resilience of organizations. It is also used as a point of reference in the deployment of flexible IDS solutions that combine machine learning and legacy security measures within actual environments.

II. LITERATURE REVIEW

Intrusion Detection Systems (IDS) are a crucial part of protecting contemporary networks from emerging cyber threats. IDS research has transformed over the years from traditional signature-based detection to more dynamic and smart mechanisms.

Liao et al. [1] provided an in-depth review of IDS technologies and classified them as signature-based, anomaly-based, and specification-based systems. They found the merits and demerits of each method and stressed the importance of hybrid IDS that use multiple detection techniques to enhance accuracy and minimize false positives.

APPENDIX 2



IEEE/CIC International Conference on Communications in China
10-13 August 2025 // Shanghai, China
Shaping the Future of Integrated Connectivity




[Home](#) • [Register](#) [Travel grants](#) [My...](#) • [Help](#)

[Home](#) » #816 (1571150910): Hybrid Intrusion Detection System: Signature Based and Anomaly Based Approach

#816 (1571150910): Hybrid Intrusion Detection System: Signature Based and Anomaly Based Approach

[Hide details](#)

twi

	Author name	Author affiliation (edit for paper)	Author email	Email	Delete
Authors	Venkata Rathnam Sunkara	University of Learning, India	vsunkarabtech21@ced.alliance.edu.in		

[📄](#) [🔍](#)

Paper title [Hybrid Intrusion Detection System: Signature Based and Anomaly Based Approach](#)

Conference and track [2025 IEEE/CIC International Conference on Communications in China \(ICCC\) - ICC25 - 11 - Communication and Information Security](#)

Abstract [✔](#) [📄](#) Intrusion Detection Systems (IDS) play an important role in detection and prevention of cyber...

Keywords [Intrusion detection](#); [signature based detection](#); [anomaly based detection](#); [cyber security](#); [hybrid cyber threat detection](#)

Topics [Communication and Information Security](#)

Personal notes [📝](#)

Roles You are the creator and an author for this paper.

Status [Active \(has manuscript\)](#)

[Review manuscript](#) [Final manuscript](#)

[📄](#) [🔍](#) [📄](#) [🔍](#)

[📄](#) [🔍](#)

up to 8 pages

APPENDIX 3

```
#window optimize script
# setup_installation.ipynb

import os
import sys
import platform
import subprocess
import json
from pathlib import Path
from IPython.display import display, Markdown
from tqdm.notebook import tqdm
import shutil

# Configuration constants
MIN_PYTHON_VERSION = (3, 8)
MIN_DISK_SPACE_GB = 10 # Increased for ELK Stack and Zeek
REQUIRED_PACKAGES = [
    'numpy>=1.20.0',
    'pandas>=1.3.0',
    'scikit-learn>=0.24.0',
    'pyshark>=0.4.5',
    'tensorflow>=2.8.0',
    'torch>=1.9.0',
    'matplotlib>=3.4.0',
    'seaborn>=0.11.0',
    'elasticsearch>=7.0.0', # For ELK Stack integration
    'snort>=2.9.0' # For signature-based detection
]

REQUIRED_DIRECTORIES = [
    'data',
    'models',
```

Fig A 3.6: Code part 1

```
'logs',
'config',
'rules', # For Snort/Suricata rules
'zeek_logs', # For Zeek Logs
'elk_stack' # For ELK Stack data

def check_system_compatibility():
    """Verify system compatibility and requirements."""
    system_info = {
        'os': platform.system(),
        'python_version': sys.version.split()[0],
        'architecture': platform.architecture()[0]
    }

    display(Markdown(f"🖥️ **System Information:**"))
    display(Markdown(f"- Operating System: {system_info['os']}"))
    display(Markdown(f"- Python Version: {system_info['python_version']}"))
    display(Markdown(f"- Architecture: {system_info['architecture']}"))

    return system_info

def check_python_version():
    """Verify Python version compatibility."""
    current = sys.version_info[:2]
    if current < MIN_PYTHON_VERSION:
        display(Markdown(f"❌ **Python {current[0]}.{current[1]} detected. Please use Python {MIN_PYTHON_VERSION[0]}.{MIN_PYTHON_VERSION[1]} or higher.**"))
        return False
    display(Markdown(f"✅ **Python version {current[0]}.{current[1]} meets requirements**"))
    return True

def check_disk_space():
    """Verify available disk space."""
    try:
```

Fig A 3.7: Code Part 2

```

try:
    if platform.system() == 'Windows':
        import psutil
        free_gb = psutil.disk_usage('C:').free // (2**30)
    else:
        free_gb = shutil.disk_usage(Path.home()).free // (2**30)

    if free_gb < MIN_DISK_SPACE_GB:
        display(Markdown(f"⚠️ **Warning: Only {free_gb}GB free disk space available. Recommend at least {MIN_DISK_SPACE_GB}GB.**"))
        return False
    display(Markdown(f"✅ **Sufficient disk space available: {free_gb}GB**"))
    return True
except Exception as e:
    display(Markdown(f"⚠️ **Warning: Could not check disk space: {str(e)}**"))
    return True

def install_package(package):
    """Install a single package with error handling."""
    try:
        subprocess.check_call(
            [sys.executable, "-m", "pip", "install", package],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.PIPE
        )
        return True, None
    except subprocess.CalledProcessError as e:
        return False, e.stderr.decode()
    except Exception as e:
        return False, str(e)

def install_requirements():
    """Install required packages with progress tracking."""
    results = {'success': [], 'failed': []}
    for package in tqdm(REQUIRED_PACKAGES, desc="Installing packages"):
        success, error = install_package(package)
        if success:
            results['success'].append(package)
            display(Markdown(f"✅ **{package} installed successfully**"))
        else:
            results['failed'].append((package, error))
            display(Markdown(f"❌ **Failed to install {package}: {error}**"))

    return results

def setup_directories():
    """Create and verify project directories."""
    base_path = Path.cwd()
    results = []

    for directory in REQUIRED_DIRECTORIES:
        dir_path = base_path / directory
        try:
            dir_path.mkdir(parents=True, exist_ok=True)
            test_file = dir_path / '.test'

            # Test write permissions
            try:
                test_file.touch()
                test_file.unlink()
                results.append((True, f"✅ **Directory verified:** `{directory}`"))
            except Exception as e:
                results.append((False, f"⚠️ **No write access to:** `{directory}`"))
        except Exception as e:
            results.append((False, f"❌ **Failed to create directory {directory}: {str(e)}**"))

    for success, message in results:
        display(Markdown(message))

```

Fig A 3.8: Code Part 3

```

for package in tqdm(REQUIRED_PACKAGES, desc="Installing packages"):
    success, error = install_package(package)
    if success:
        results['success'].append(package)
        display(Markdown(f"✅ **{package} installed successfully**"))
    else:
        results['failed'].append((package, error))
        display(Markdown(f"❌ **Failed to install {package}: {error}**"))

return results

def setup_directories():
    """Create and verify project directories."""
    base_path = Path.cwd()
    results = []

    for directory in REQUIRED_DIRECTORIES:
        dir_path = base_path / directory
        try:
            dir_path.mkdir(parents=True, exist_ok=True)
            test_file = dir_path / '.test'

            # Test write permissions
            try:
                test_file.touch()
                test_file.unlink()
                results.append((True, f"✅ **Directory verified:** `{directory}`"))
            except Exception as e:
                results.append((False, f"⚠️ **No write access to:** `{directory}`"))
        except Exception as e:
            results.append((False, f"❌ **Failed to create directory {directory}: {str(e)}**"))

    for success, message in results:
        display(Markdown(message))

```

Fig A 3.9: Code Part 4


```

    return all(result[0] for result in results)

def create_config():
    """Create default configuration file."""
    config = {
        'interface': 'Ethernet', # Default for Windows
        'log_level': 'INFO',
        'data_directory': str(Path.cwd() / 'data'),
        'model_directory': str(Path.cwd() / 'models'),
        'max_packet_count': 1000,
        'capture_timeout': 60,
        'snort_rules_path': str(Path.cwd() / 'rules' / 'snort.rules'),
        'zeek_logs_path': str(Path.cwd() / 'zeek_logs'),
        'elk_stack_path': str(Path.cwd() / 'elk_stack')
    }

    config_path = Path.cwd() / 'config' / 'config.json'
    try:
        with open(config_path, 'w') as f:
            json.dump(config, f, indent=4)
        display(Markdown(f"✅ **Configuration file created at** `{config_path}`"))
        return True
    except Exception as e:
        display(Markdown(f"❌ **Failed to create configuration file: {str(e)}**"))
        return False

def main():
    """Main setup execution function."""
    display(Markdown("🚀 **Starting setup...**\n"))

    # Check system compatibility
    system_info = check_system_compatibility()

```

Fig A 3.10: Code Part 5

```

# verify python version
if not check_python_version():
    return False

# Check disk space
if not check_disk_space():
    if not input("Continue anyway? (y/n): ").lower().startswith('y'):
        return False

# Setup directories
display(Markdown("\n📁 **Setting up project structure...**"))
if not setup_directories():
    return False

# Create configuration
display(Markdown("\n⚙️ **Creating configuration...**"))
if not create_config():
    return False

# Install requirements
display(Markdown("\n📦 **Installing required packages...**"))
results = install_requirements()

# Final status
if not results['failed']:
    display(Markdown("\n🎉 **Setup completed successfully!**"))
    return True
else:
    display(Markdown("\n⚠️ **Setup completed with some issues. Please check the logs above.**"))
    return False

if __name__ == "__main__":
    main()

```

Fig A 3.11: Code Part 6

References

- [1] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, “Intrusion detection system: A comprehensive review,” *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013, doi: 10.1016/j.jnca.2012.09.004.
- [2] S. Ahmadi, “Network Intrusion Detection in Cloud Environments: A Comparative Analysis of Approaches,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 3, pp. 1–7, 2024.
- [3] S. Racherla, S. P. Sahu, and A. S. Rajawat, “Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning,” *IEEE Access*, vol. 12, pp. 42615–42628, 2024, doi: 10.1109/ACCESS.2024.3342798.
- [4] I. Mudraje, A. V. Kumar, and K. S. Bhat, “Design of an Autonomous Intrusion Classification Device for FIDS Robustness,” *IEEE Access*, vol. 12, pp. 50467–50476, 2024, doi: 10.1109/ACCESS.2024.3375432.
- [5] H. Alzahrani, T. Sheltami, A. Barnawi, M. Imam, and A. Yaser, “A Lightweight Intrusion Detection System Using Convolutional Neural Network and Long Short-Term Memory in Fog Computing,” *Computers, Materials & Continua*, vol. 80, no. 3, pp. 4703–4718, 2024, doi: 10.32604/cmc.2024.054203.
- [6] G. Galey, “Intrusion Detection Systems for In-Vehicle Networks,” in *Proc. IEEE 91st Vehicular Technology Conference (VTC)*, pp. 91–96, 2024.
- [7] D. E. Denning, “An Intrusion-Detection Model,” *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987, doi: 10.1109/TSE.1987.232894.
- [8] A. Patcha and J.-M. Park, “An Overview of Anomaly Detection Techniques: Existing Solutions and Latest Technological Trends,” *Computer Networks*, vol. 51, no. 12, pp. 3448–3470, 2007, doi: 10.1016/j.comnet.2006.11.001.

- [9] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A Detailed Analysis of the KDD CUP 99 Data Set,” in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.
- [10] N. Hubballi and V. Suryanarayanan, “False Alarm Minimization Techniques in Signature-Based Intrusion Detection Systems: A Survey,” *Computer Communications*, vol. 49, pp. 1–17, Aug. 2014, doi: 10.1016/j.comcom.2014.04.011.
- [11] M. Elsaid, R. A. Abdelrahman, and A. M. Mostafa, “A Novel Hybrid Intrusion Detection Model Based on Ensemble Feature Selection and Classification for DDoS Attack Detection,” *Multimedia Tools and Applications*, vol. 83, pp. 29377–29398, 2024, doi: 10.1007/s11042-024-17838-x.
- [12] F. A. Hussein, M. M. Abdelwahab, and H. F. Eid, “Hybrid Deep Learning Model for Network Intrusion Detection in Cloud Computing,” *Journal of Cloud Computing*, vol. 13, no. 1, pp. 1–16, 2024, doi: 10.1186/s13677-024-00527-4.
- [13] R. Kushal, S. P. Sethi, and D. P. Mohapatra, “Self-Healing Hybrid Intrusion Detection System: An Ensemble Machine Learning Approach,” *Wireless Personal Communications*, vol. 132, pp. 1983–2001, 2024, doi: 10.1007/s11277-024-10831-6.
- [14] F. Amouri, H. Guesmi, and M. M. Nasr, “KAN-XGBoost: A Hybrid Intrusion Detection Approach Using Kolmogorov-Arnold Networks and XGBoost,” *arXiv preprint*, arXiv:2408.15886, Aug. 2024.
- [15] A. S. Akif, M. R. Haider, and M. H. Kabir, “A Robust Hybrid Machine Learning-Based Intrusion Detection Framework for IoT Networks,” *arXiv preprint*, arXiv:2502.12382, Feb. 2025.